Oriented Object Detection

Oriented Object Detection

1. Course Content
2. Preparation
3. Case Demonstration
    3.1 YOLO Pre-trained Model Detection
    3.2 OBB Waste Detection Model
4. Source Code Analysis

1. Course Content

- Use the YOLO OBB detection model to detect objects in the camera image and display the class and confidence.
- Use the factory-pre-trained waste detection model to detect the oriented bounding boxes of the simulated waste blocks.
- Oriented object detection goes a step further than standard object detection by introducing an extra angle to more accurately locate objects in the image. The output of an oriented object detector is a set of rotated bounding boxes that precisely enclose objects in the image, while providing a class label and confidence score for each box.

2. Preparation

- Demonstration props
    - 3 * 3 * 3 cm waste label block



3. Case Demonstration

3.1 YOLO Pre-trained Model Detection

```
ros2 launch yahboomcar_bringup camera_demo.launch.py demo:=yolo_obb
```

The detection image consists of the following elements:

- **Oriented Bounding Box**: The model draws a rectangular box with a rotation angle for each detected target; the orientation of the box is consistent with the actual direction of the target, fitting the target contour more tightly than an ordinary horizontal box.

- **Class name and confidence**: Next to each oriented box, the target class name (such as large vehicle, small vehicle) and the confidence (0~1) are labeled; a higher confidence means the model is more certain.
- **Upper-left status bar**: Green text displays the real-time frame rate FPS, and yellow text displays the single-frame inference time (milliseconds).



3.2 OBB Waste Detection Model

```
ros2 launch yahboomcar_bringup camera_demo.launch.py demo:=waste_detect
```

- You can see that while detecting the waste block target, the image also shows the rotation angle information of the target. For more details, refer to the tutorial 【OpenClaw Embodied Intelligence in Practice — OpenClaw Waste Sorting】.



- The detection information of the target is published on the `/waste_detections` topic.

```
ros2 topic echo /waste_detections --once
```

```
jetson@yahboom:~$ ros2 topic echo /waste_detecttions --once
header:
  stamp:
    sec: 1789441488
    nanosec: 738845250
  frame_id: ''
waste_detections:
- class_name: cigarette-butts
  classification: OtherWaste
  confidence: 0.9058337807655334
  center_x: 195.46170043945312
  center_y: 273.665283203125
  width: 49.097633361816406
  height: 35.901451110839844
  angle: -0.5978236198425293
  points:
  - x: 225.85630798339844
    y: 274.6855163574219
  - x: 205.6493682861328
    y: 245.01071166992188
  - x: 165.0670928955078
    y: 272.6450500488281
  - x: 185.27403259277344
    y: 302.3198547363281
```

Topic data meaning description:

Field	Type	Meaning description
`header.stamp`	time	Timestamp of the detection result (sec seconds, nanosec nanoseconds)
`waste_detections`	WasteItem[]	List of detected waste targets (may contain multiple targets)
`class_name`	string	The specific class name of the target (such as cigarette-butts, batteries, etc.)
`classification`	string	Waste classification type (Recyclable, KitchenWaste, OtherWaste, HazardousWaste)
`confidence`	float	Detection confidence (0.0~1.0); a higher value means the model is more certain of the detection result
`center_x`	float	X coordinate of the center point of the oriented box (in pixels, relative to the upper left corner of the image)
`center_y`	float	Y coordinate of the center point of the oriented box (in pixels, relative to the upper left corner of the image)
`width`	float	Width of the oriented box (in pixels)
`height`	float	Height of the oriented box (in pixels)
`angle`	float	Rotation angle of the oriented box (in radians), a parameter unique to the OBB model, representing the orientation of the object
`points`	Point2D[]	The 4 vertex coordinates of the oriented box, used to precisely describe the object boundary

Key concept description:

1. **OBB (Oriented Bounding Box)**: Unlike the traditional horizontal rectangular box, an OBB box can rotate with the direction of the object, fitting the actual contour of irregular objects more tightly.
2. **angle**: Represents the rotation angle of the object relative to the horizontal direction; the robotic arm adjusts the gripper direction accordingly when grasping, ensuring grasping from the optimal angle.
3. **points vertex coordinates**: The 4 vertices precisely describe the boundary of the oriented box and can be used to calculate the actual size and orientation of the object.

4. Source Code Analysis

- Source code path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/yolo/yolo_obb.py
```

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/waste_classify/waste_classify/waste_classify.py
```

The following code fragment is taken from the `YOLO_OBB` class of `yolo_obb.py`; `waste_classify.py` uses the same `task='obb'` inference pipeline and packages the detection results of each frame (`result.obb.xywhr`, `xyxyxyxy`, `cls`, `conf`) into custom messages published to the `/waste_detections` topic, for use by the upper-level robotic arm grasping logic.

Constructor description:

- `declare_parameter('init_pose', [90, 180, 0, 0, 90, 30])`: Declares the robot arm initial pose parameter, 6 joint angle values (degrees), which are arm1~arm5 and the gripper angle in order.
- `declare_parameter('model_path', '')`: Declares the YOLO model file path parameter, defaulting to empty; the model file must be specified via the `-p` command line.
- `declare_parameter('max_det', 20)`: Declares the maximum number of detected objects per frame; for the OBB task, at most 20 objects are detected by default.
- `create_client(ArmPosition, 'set_arm_joints')`: Creates a robot arm joint control service client, used to move the robot arm to the initial pose.
- `CvBridge()`: Creates a bridge for conversion between ROS image messages and OpenCV images.
- `create_subscription(Image, 'camera_rgb_topic', ...)`: Subscribes to the camera RGB image topic; the callback is triggered each time an image frame is received.
- `YOLO(self.model_path, task='obb')`: Loads the YOLO model, specifying the task type as `obb` (oriented object detection).
- `_init_arm_pose()`: Moves the robot arm to the initial pose after the model is loaded.

_init_arm_pose function description:

- `wait_for_service(timeout_sec=10.0)`: Waits for the robot arm joint control service to come online; if it times out after 10 seconds, the initial pose setup is skipped.
- `ArmPosition.Request()`: Constructs a joint control request, where `request_joints` is the initial pose angle list.
- `call_async(req)`: Sends the joint control request asynchronously.
- `add_done_callback(self._init_arm_done_callback)`: Registers a callback function to check the execution result after the request completes.

get_RGBImageCallBack function description:

- `imgmsg_to_cv2(rgb_msg, desired_encoding='bgr8')`: Converts the ROS Image message into an OpenCV BGR format image.
- `self.yolo_model.predict(rgb_image, stream=True, max_det=self.max_det, verbose=False, conf=0.1)`: Performs YOLO oriented object detection inference on the current frame; `conf=0.1` sets the confidence threshold to 0.1 to display more low-confidence targets (for demonstration), and `stream=True` enables streaming output to save memory.

- `res.obb`: The result object of the OBB task; a non-empty value indicates that a target was detected in this frame, containing attributes such as `xywhr` (center x, y, width, height, rotation angle), `xyxyxyxy` (the 4 vertex coordinates of the oriented box), `cls` (class ID), and `conf` (confidence).
- `res.plot().copy()`: Draws the oriented box, class name, and confidence onto the image; `.copy()` creates an image copy to avoid modifying the original result object when subsequently overlaying FPS text.
- FPS calculation: Counts the number of frames within 1 second to calculate the real-time frame rate.
- `cv2.putText(...)`: Overlays and displays the FPS and the single-frame inference time (milliseconds) in the upper left corner of the image.
- `cv2.imshow(...)` / `cv2.waitKey(1)`: Displays the detection image; `waitKey(1)` refreshes the window.

```
class YOLO_OBB(Node):
    def __init__(self,name):
        super().__init__(name)
        self.declare_parameter('init_pose', [90, 180, 0, 0, 90, 30])
        self.init_pose =
self.get_parameter('init_pose').get_parameter_value().integer_array_value
        self.declare_parameter('model_path','')
        self.model_path =
self.get_parameter('model_path').get_parameter_value().string_value
        self.declare_parameter('max_det', 20)
        self.max_det =
self.get_parameter('max_det').get_parameter_value().integer_value
        self.set_arm_client = self.create_client(ArmPosition, 'set_arm_joints')
        self.rgb_bridge = CvBridge()
        self.msg2img_bridge = CvBridge()
        self.count=0
        self.sub_rgb =
self.create_subscription(Image,'camera_rgb_topic',self.get_RGBImageCallBack,1)
        self.frame_count=0
        self.prev_time = time.time()
        self.frame_count = 0
        self.fps = 0.0

        self.yolo_model = YOLO(self.model_path, task='obb')
        self.get_logger().info(Fore.GREEN + 'YOLOV26-OBB Model Loaded.' +
Fore.RESET)
        self._init_arm_pose()

    def _init_arm_pose(self):
        if not self.set_arm_client.wait_for_service(timeout_sec=10.0):
            self.get_logger().warn("Failed to connect to set_arm_joints server,
skipping init arm pose")
            return
        req = ArmPosition.Request()
        req.request_joints = list(self.init_pose)
        future = self.set_arm_client.call_async(req)
        future.add_done_callback(self._init_arm_done_callback)
```

```
    def _init_arm_done_callback(self, future):
        result = future.result()
        if not result.success:
            self.get_logger().warn(f"Init arm pose failed:
{list(result.result_joints)}, msg: {result.message}")


    def get_RGBImageCallBack(self,rgb_msg):
        rgb_image = self.rgb_bridge.imgmsg_to_cv2(rgb_msg,
desired_encoding='bgr8')
        infer_start = time.time()
        results = self.yolo_model.predict(rgb_image, stream=True,
max_det=self.max_det, verbose=False,conf=0.1)
        for res in results:
            if res.obb is not None:
                annotated_frame = res.plot().copy()
        infer_end = time.time()
        self.frame_count += 1
        curr_time = time.time()
        time_diff = curr_time - self.prev_time
        if time_diff >= 1.0:
            self.fps = self.frame_count / time_diff
            self.frame_count = 0
            self.prev_time = curr_time
        cv2.putText(annotated_frame,f"FPS: {self.fps:.2f}",(10,
25),cv2.FONT_HERSHEY_SIMPLEX,0.6,(0, 255, 0),1,)
        infer_time = (infer_end - infer_start) * 1000
        cv2.putText(annotated_frame, f"Infer: {infer_time:.1f} ms", (10,
45),cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 1)

        cv2.imshow("YOLO26-OBB Panel", annotated_frame)
        cv2.waitKey(1)
```